

Theoretical questions:

1. In pure functional programming there are no side effects (state changes). Therefore any expression can be evaluated by evaluating multiple functions one at a time, while passing the other expressions as parameters for it. In Imperative languages on the other hand, multiple expressions are useful inside the function body. L3 is a functional programming language, therefore, the former describes L3 better.

2. a) Special forms are required in programming languages because they allow the use of parameters that cannot be evaluated at the time of execution or choose which parameters to evaluate based on the result of other expressions. Primitive operators on the other hand, evaluate all the parameters so they don't allow operations such as variable binding or condition statements. For example the "if" operator only evaluates the parameter in the location that matches the condition statement value.

b) The 'or' operator can be implemented as a primitive operator, but in order to apply shortcut semantics (a way to avoid evaluating a parameter in case it doesn't change the result) it must be defined as a special form.

3. a) The result would be 3, because in 'let', the initial values of the bindings are computed before any of the variables becomes bound, therefore the interpreter will use the outer value of x (which is 1) while evaluating y.

b) The program will be evaluated to be 15, because in 'let', the bindings and evaluations are performed sequentially from left to right, thus the new value of x is bound when binding y.

c) `(define x 1) (let ((x 5))(let ((y (* x 3))) y))`

d) `(define x 1) ((lambda (x) ((lambda (y) y) (* x 3))) 5)`

4. a) The valueToLitExp function would convert a literal to the corresponding AST node during the compilation process.

b) Because the AST is already formed in the parsing phase.

c) Because literal values are directly embedded in the AST nodes during parsing, then the interpreter evaluates the AST nodes according to the bindings in the environment.

5. a) We might want to switch to normal evaluation in case we want to avoid pointless calculations, for example when evaluating an "if" expression, there's no need to evaluate both parameters - only the one that will be returned.

b) We might want to switch to applicative evaluation to make debugging easier or to better control the order of evaluation of certain expressions. For example when

debugging a recursive function, it's better to see the values of the parameters in each recursive step to distinguish the current stage of the program.

6. a) Because the variables are encapsulated inside of an object that represents the environment, and therefore there is no problem with variables to have the same name, because they are addressed differently.

b) Renaming is typically not required for a "closed" term because there are no variable conflicts to resolve. In addition, Substitution directly replaces variables with their corresponding expressions, and with closed terms, all variables are bound within their scope.